

Node.js & Express.js — Learning Notes

Topics covered so far: npm, package.json, npm init, local vs global packages, Git Bash project workflow, Express routing, requests, and responses.

1. What is npm?

npm is the Node Package Manager. It is used to manage JavaScript/Node.js packages and project dependencies.

```
npm install <package>
npm init
npm install -g <package>
```

2. What is package.json?

package.json is the configuration and metadata file for a Node.js project. It describes the project, scripts, dependencies, and other information.

```
{
  "name": "figlet",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "..."
  },
  "type": "commonjs",
  "dependencies": {
    "figlet": "^1.11.4"
  }
}
```

Important fields

Field	Purpose
name	Project name
version	Project version
main	Main/entry JavaScript file
scripts	Commands that can be run with npm
type	Determines the JavaScript module system
dependencies	Packages required by the project

3. npm init

npm init initializes an npm project and creates a package.json file.

```
npm init
```

npm init -y initializes the project using default answers without asking the setup questions.

```
npm init -y
```

4. npm install

npm install downloads a package and makes it available to your project. For example, in the Figlet project:

```
npm install figlet
```

This normally creates/updates:

```
Figlet/  
■■■ node_modules/  
■■■ package.json  
■■■ package-lock.json
```

5. Local vs Global Packages

Local package: installed for a specific project.

```
npm install figlet
```

The package is placed in the project's node_modules and recorded in package.json.

Global package: installed for use across the environment, commonly for command-line tools.

```
npm install -g <package>
```

The **-g** means global. For normal application dependencies, local installation is usually preferred.

6. CommonJS vs ES Modules

Your current package.json contains:

```
"type": "commonjs"
```

With CommonJS, you normally use **require()**:

```
const figlet = require("figlet");
```

If the project uses **type: module**, you can use ES module syntax such as **import**:

```
import figlet from "figlet";
```

7. Running a JavaScript File

From inside the project folder, run:

```
node index.js
```

You can also create an npm script such as:

```
"scripts": {  
  "start": "node index.js"  
}
```

Then run:

```
npm start
```

8. Git Bash Navigation

```
pwd          # Show current directory
```

```
ls          # List files/folders
cd folder   # Enter a folder
cd ..       # Go back one folder
cd ../..    # Go back two folders
cd ~        # Go to home directory
```

9. Express.js Routing

Routing means deciding what code should run when a client sends a request to a particular path and HTTP method.

```
app.get("/", (req, res) => {
  res.send("Welcome to my website");
});
```

Here, **GET** is the HTTP method, */* is the route/path, and the callback is the route handler.

10. Request and Response

req means request. It contains information coming from the client.

```
req.url
req.method
req.params
req.query
req.body
```

res means response. It is used to send information back to the client.

```
res.send()
res.json()
res.status()
```

11. Routing + Response Example

```
app.get("/products", (req, res) => {
  res.json({
    product: "Laptop",
    price: 1000
  });
});
```

The flow is:

```
Client
↓
GET /products
↓
Express routing
↓
Route handler
↓
res.json(...)
↓
Client receives response
```

12. Key Takeaways

```
npm init      → initializes the project
package.json  → project configuration/metadata
npm install   → installs a package
```

node_modules	→ installed packages
package-lock	→ records exact dependency versions
-g	→ global installation
require()	→ CommonJS import style
node index.js	→ runs a JavaScript file
routing	→ maps requests to handlers
req	→ incoming request information
res	→ outgoing response